



Benchmark Tests

Measure Performance.
Optimize with Confidence.



Chapter 5 • Go Testing Handbook



Correct ≠ Fast



Your code can **pass every test...** and still be too slow for production.



Benchmark Tests answer:

“How **fast** is my code?”



```
go test -bench=. -benchmem
```

```
BenchmarkMyFunc-8      10000000      72.3 ns/op      0 B/op      0 allocs/op
```

```
PASS
```





Unit Test vs Benchmark



Unit Test

Is it **correct**?



Benchmark

How **fast** is it?

VS



Both matter.

They answer different questions.





Writing a Benchmark

Every benchmark:



Starts with
Benchmark



Uses
***testing.B**



Lives in a
_test.go file



Simple structure.
Powerful insights.





Meet **b.N**



You never choose **b.N**.



Go automatically increases the number of iterations until it collects **stable** and **reliable** performance measurements.



```
func BenchmarkMyFunc(b *testing.B) {  
    for i := 0; i < b.N; i++ {  
        MyFunc()  
    }  
}
```



b.N Grows Automatically

Go runs your benchmark multiple times, each time increasing **b.N**...



Stops when results are **stable** and **reliable**.





Run & Measure



Run every **benchmark** with:



```
$ go test -bench=.
```



BenchmarkMyFunc-8	1000000	128 ns/op
BenchmarkOther-8	2000000	65 ns/op



Need **memory** statistics too?

```
$ go test -bench=. -benchmem
```



BenchmarkMyFunc-8	1000000	128 ns/op	48 B/op	2 allocs/op
BenchmarkOther-8	2000000	65 ns/op	16 B/op	1 allocs/op



Small changes. Big impact.
Measure, don't guess.





Read the Results

Learn what each **metric** means:



Benchmark	ns/op	B/op	allocs/op
BenchmarkMyFunc-8	128 ns/op	48 B/op	2 allocs/op
BenchmarkOther-8	65 ns/op	16 B/op	1 allocs/op



ns/op

Nanoseconds per operation.
Lower is better.



B/op

Bytes allocated per operation.
Lower is better.



allocs/op

Number of allocations per operation.
Lower is better.



Measure. **Compare.** Optimize.





Benchmark Before You Optimize

GO

Never rely on **assumptions**.
Use benchmarks to compare
implementations, reduce
allocations, and make
data-driven performance
decisions.

```
go test -bench=. -benchmem
```

Benchmark	ns/op	B/op	allocs/op
BenchmarkMyFunc-8	128 ns/op	48 B/op	2 allocs/op
BenchmarkOther-8	65 ns/op	16 B/op	1 allocs/op

**Measure first.****Optimize second.** 🚀

Read the full chapter in the
Go Testing Handbook 📖
on GitHub



REPOSITORY

github.com/
golang-evangelist/handbook/.../05-benchmarks.md

**Measure. Compare. Optimize.**

Data-driven code is high-performance code.